

FullStack.Cafe - Kill Your Tech Interview

Q1: What is an Activity ? ☆

Topics: Android

Answer:

An **Activity** provides the window in which the app draws its UI.

This window typically fills the screen, but may be smaller than the screen and float on top of other windows. Generally, **one activity implements one screen** in an app. For instance, one of an app's activities may implement a Preferences screen, while another activity implements a Select Photo screen.

Q2: What is a difference between Spannable and String ? ☆

Topics: Android

Answer:

A **Spannable** allows to attach formatting information like bold, italic, ... to sub-sequences ("spans", thus the name) of the characters. It can be used whenever you want to represent **rich text**.

Q3: What is the difference between Bitmap and Drawable in Android? ☆

Topics: Android

Answer:

- A **Bitmap** is a representation of a bitmap image (something like java.awt.Image).
- A **Drawable** is an abstraction of "something that can be drawn". It could be a Bitmap (wrapped up as a BitmapDrawable), but it could also be a solid color, a collection of other Drawable objects, or any number of other structures.

Q4: Mention the difference between RelativeLayout and LinearLayout ? ☆

Topics: Android

Answer:

- **Linear Layout** — Arranges elements either vertically or horizontally. i.e. in a row or column.
- **Relative Layout** — Arranges elements relative to parent or other elements.

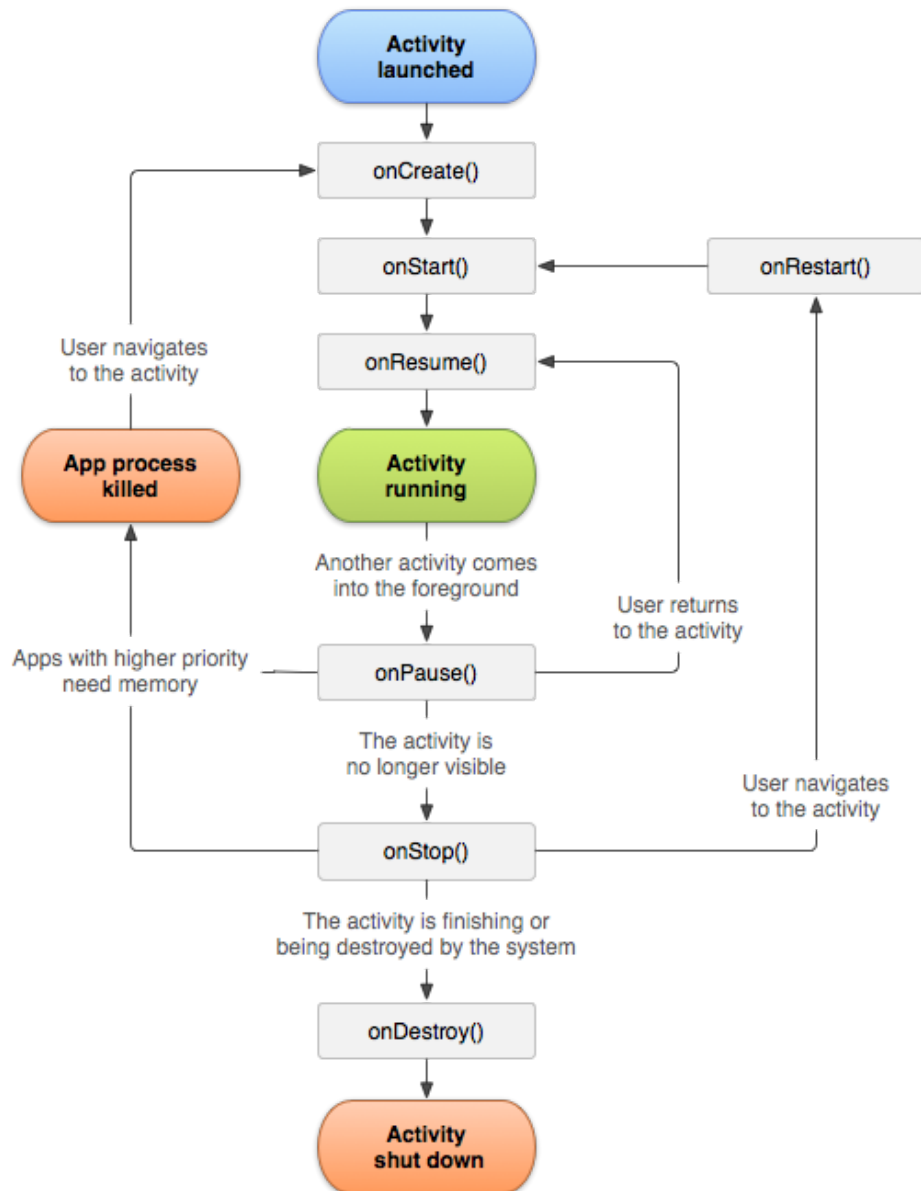
Q5: Explain activity lifecycle ☆☆

Topics: Android

Answer:

As a user navigates through, out of, and back to your app, the Activity instances in your app transition through different states in their lifecycle.

To navigate transitions between stages of the activity lifecycle, the Activity class provides a core set of six callbacks: `onCreate()`, `onStart()`, `onResume()`, `onPause()`, `onStop()`, and `onDestroy()`. The system invokes each of these callbacks as an activity enters a new state.

**Q6: Explain briefly all the Android application components ☆☆**

Topics: Android

Answer:

App components are the essential building blocks of an Android app. Each component is an entry point through which the system or a user can enter your app.

There are four different types of app components:

- **Activities** - An activity is the entry point for interacting with the user. It represents a single screen with a user interface.
- **Services** - A service is a general-purpose entry point for keeping an app running in the background for all kinds of reasons. It is a component that runs in the background to perform long-running operations or to perform work for remote processes.
- **Broadcast receivers** - A broadcast receiver is a component that enables the system to deliver events to the app outside of a regular user flow, allowing the app to respond to system-wide broadcast announcements.
- **Content providers** - A content provider manages a shared set of app data that you can store in the file system, in a SQLite database, on the web, or on any other persistent storage location that your app can access.

Q7: How do I pass data between Activities in Android application?

☆☆

Topics: Android

Problem:

I have a scenario where, after logging in through a login page, there will be a sign-out button on each activity. Can you guide me on how to keep session id available to all activities?

Solution:

The easiest way to do this would be to pass the session id to the signout activity in the **Intent** you're using to start the activity:

```
Intent intent = new Intent(getBaseContext(), SignoutActivity.class);
intent.putExtra("EXTRA_SESSION_ID", sessionId);
startActivity(intent);
```

Access that intent on next activity:

```
String sessionId = getIntent().getStringExtra("EXTRA_SESSION_ID");
```

Q8: What is an Intent in Android? ☆☆

Topics: Android

Answer:

An **Intent** is basically a message that is passed between components (such as **Activities**, **Services**, **Broadcast Receivers**, and **Content Providers**). So, it is almost equivalent to parameters passed to API calls. The fundamental differences between API calls and invoking components via intents are:

- API calls are synchronous while intent-based invocations are asynchronous.
- API calls are compile-time binding while intent-based calls are run-time binding.

To listen for an broadcast intent (like the phone ringing, or an SMS is received), you implement a **broadcast receiver**, which will be passed the intent. To declare that you can handle another's app intent like "take picture", you declare an intent filter in your app's manifest file.

If you want to fire off an intent to do something, like pop up the dialer, you fire off an intent saying you will.

An Intent provides a facility for performing late runtime binding between the code in different applications.

Q9: What is the most appropriate way to store user settings in Android application? ☆☆

Topics: Android

Answer:

In general **SharedPreferences** are your best bet for storing preferences, so in general I'd recommend that approach for saving application and user settings.

The only area of concern here is what you're saving. Passwords are always a tricky thing to store, and I'd be particularly wary of storing them as clear text. The Android architecture is such that your application's SharedPreferences are sandboxed to prevent other applications from being able to access the values so there's some security there, but physical access to a phone could potentially allow access to the values.

Q10: In what situation should one use **RecyclerView** over **ListView**? ☆☆

Topics: Android

Answer:

RecyclerView was created as a **ListView** improvement, so yes, you can create an attached list with **ListView** control, but using **RecyclerView** is easier as it:

- Reuses cells while scrolling up/down - this is possible with implementing View Holder in the ListView adapter, but it was an optional thing, while in the RecyclerView it's the default way of writing adapter.
- Decouples list from its container - so you can put list items easily at run time in the different containers (LinearLayout, GridLayout) with setting LayoutManager.

To conclude, **RecyclerView** is a more flexible control for handling "list data" that follows patterns of delegation of concerns and leaves for itself only one task - recycling items.

Q11: What is *Dalvik*? ☆☆

Topics: Android

Answer:

Dalvik is a **Just In Time (JIT)** compiler.

By the term JIT, we mean to say that whenever you run your app in your mobile device then that part of your code that is needed for execution of your app will only be compiled at that moment and rest of the code will be compiled in the future when needed. The JIT or Just In Time compiles only a part of your code and it has a smaller memory footprint and due to this, it uses very less physical space on your device.

Q12: How to persist data in an Android app? ☆☆

Topics: Android

Answer:

There are basically four different ways to store data in an Android app:

1. Shared Preferences - to save primitive data in key-value pairs
2. Internal Storage - you need to store data to the device filesystem, but you do not want any other app (even the user) to read this data
3. External Storage - you might want the user to view the files and data saved by your app
4. SQLite database

Q13: Explain Android notification system ☆☆

Topics: Android

Answer:

A **notification** is a message that Android displays outside your app's UI to provide the user with reminders, communication from other people, or other timely information from your app. Users can tap the notification to open your app or take an action directly from the notification.

Notifications appear to users in different locations and formats, such as an icon in the status bar, a more detailed entry in the notification drawer, as a badge on the app's icon, and on paired wearables automatically. Beginning with Android 5.0, notifications can appear on the lock screen.

Starting in Android 8.0 (API level 26), all notifications must be assigned to a channel or it will not appear. By categorizing notifications into channels, users can disable specific notification channels for your app (instead of disabling all your notifications), and users can control the visual and auditory options for each channel—all from the Android system settings.

Q14: Why is it recommended to use only the *default constructor* to create a `Fragment` ? ☆☆

Topics: Android

Answer:

In short, **Fragments** need to have a no-args constructor for the Android system to instantiate them. Your **Fragment** subclasses need a public empty constructor as this is what's being called by the framework.

It is used in the case when device has to restore the state of a fragment. No data will be passed and a default fragment will be created and then the state will be restored. Since the system has no way to know what you passed in your constructor or your `newInstance`, default constructor will be used and saved bundle should be passed via `onCreate` after the fragment is actually instantiated with the default constructor.

Q15: What is an `AsyncTask` ? ☆☆

Topics: Android

Answer:

`AsyncTask` is one of the easiest ways to implement parallelism in Android without having to deal with more complex methods like `Threads`. Though it offers a basic level of parallelism with the UI thread, it should not be used for longer operations (of, say, not more than 2 seconds).

`AsyncTask` has four methods

- `onPreExecute()`
- `doInBackground()`
- `onProgressUpdate()`
- `onPostExecute()`

where `doInBackground()` is the most important as it is where background computations are performed.

Q16: What is a `ContentProvider` and what is it typically used for? ☆☆

Topics: Android

Answer:

A **content provider** manages access to a central repository of data. A provider is part of an Android application, which often provides its own UI for working with the data. However, content providers are primarily intended to be used by other applications, which access the provider using a provider client object.

Typically you work with content providers in one of two scenarios;

- you may want to implement code to **access an existing content provider** in another application, or
- you may want to **create a new content provider** in your application to share data with other applications.

Q17: What is **ADB** and what is it used for? ☆☆

Topics: Android

Answer:

ADB is the acronym for **Android Debug Bridge**, which is part of the Android SDK (Software Development Kit). It uses a client-server-model (i.e. `adb`, the ADB daemon, is running on the device and can be connected to), and in most cases is used via an USB connection. It is also possible to use it via WiFi (wireless adb).

There's nothing you need to install on your Android device, as the ADB daemon (`adb`) is already integrated into the Android OS. It is usually accessed via a command line interface from the PC, where either the full Android SDK is installed (several 30 MB download archive currently), or a massively stripped-down version for "non-developers", sometimes referred to as "Mini ADB" or "ADB essentials" (for Linux, this is only the `adb` executable; for Windows it's `adb.exe` plus two or three `.dll` files).

Q18: Explain the build process in Android ☆☆

Topics: Android

Answer:

1. First step involves compiling the resources folder (`/res`) using the `aapt` (android asset packaging tool) tool. These are compiled to a single class file called `R.java`. This is a class that just contains constants.
2. Second step involves the java source code being compiled to `.class` files by `javac`, and then the class files are converted to Dalvik bytecode by the "dx" tool, which is included in the sdk 'tools'. The output is `classes.dex`.
3. The final step involves the android `apkbuilder` which takes all the input and builds the apk (android packaging key) file.

Q19: What's the difference between `onCreate()` and `onStart()` ? ☆☆

Topics: Android

Answer:

- The `onCreate()` method is called once during the **Activity** lifecycle, either when the application **starts**, or when the Activity has been destroyed and then **recreated**, for example during a configuration change.
- The `onStart()` method is called whenever the **Activity** becomes **visible** to the user, typically after `onCreate()` or `onRestart()`.

Q20: What is *View Group*? How are they different from *Views*? ☆☆

Topics: Android

Answer:

- **View:** `View` objects are the basic building blocks of User Interface(UI) elements in Android. View is a simple rectangle box which responds to the user's actions. Examples are `EditText`, `Button`, `CheckBox` etc. View refers to the `android.view.View` class, which is the base class of all UI classes.
- **ViewGroup:** `ViewGroup` is the invisible container. It holds `View` and `ViewGroup`. For example, `LinearLayout` is the `ViewGroup` that contains `Button` (View), and other Layouts also. `ViewGroup` is the base class for Layout S.